

PRM3

Théorie : Leçon I

Java :

Entités, types, expressions et assignations

Entités

Variable : sa valeur peut changer.

Type

Nom

Espace mémoire

Valeur

Littéral : sa valeur est fixe.

Type

Valeur

Exemples : `15`, `true`, `'b'`, `' '`, `"bonjour"`, `3.14`

Variables

Type : définit le type de valeur contenu à l'espace mémoire spécifié.

Entier (`int`)

Réel (`double`)

Booléen (`boolean`)

Caractère (`char`)

...

Espace mémoire : endroit dans la mémoire centrale où est stockée la valeur de la variable.

Nom : permet d'identifier et de manipuler (lire/écrire) la valeur écrite à l'espace mémoire spécifié.

Par exemple : `tva`, `compteur`, `resultat`,...

Déclarations de variables

Type et nom

Actions :

Réserve un **espace mémoire** de taille spécifiée par le type

Lie le nom à cet espace mémoire

Exemples :

```
int compteur; boolean b; double tva; char c;
```

La déclaration d'une variable se fait toujours avant son utilisation.

Après sa déclaration, la variable n'a pas de valeur. Il faudra lui en donner une avant de l'utiliser.

Choisir un nom valable et significatif. Attention à la casse.

Opérateurs arithmétiques

+

–

*

/ : division (entière ou réelle)

5 / 2 donne 2

5 / 2.0 ou 5.0 / 2 ou 5.0 / 2.0 donnent 2.5

% : modulo, le reste de la division entière

5 % 2 donne 1

78 % 60 donne 18

Expression

Une expression dénote une valeur.

Elle est formée d'**opérateurs** et d'**opérandes**.

Exemples :

a

3

$a * 3$

$(3 + a) * b$

On peut composer de nouvelles expressions à partir d'expressions et d'opérateurs.

Valeur d'une expression

La valeur d'une expression est calculée à partir des valeurs des variables **au moment de l'évaluation**.

Règles d'évaluation :

- Priorités des opérateurs (* / % puis + -)

- Parenthèses

- Ordre d'évaluation de gauche à droite

Assignment

```
<variable> = <expression> ;
```

“=” se dit “reçoit”

Actions :

évaluer l’expression

écrire le résultat de l’évaluation dans l’espace mémoire de la variable.

Le type de l’expression doit être compatible avec le type de la variable

Attention :

aux variables déclarées mais non-initialisées

`a = b` n’est pas `a == b`

Déclarer et initialiser une variable : `int a = 3;`

Assignation

Assignation composée :

Raccourcis d'écriture

$a += 3$ équivaut à $a = a + 3$

$a *= 2$ équivaut à $a = a * 2$

$a *= 1 + 2$ équivaut à $a = a * (1 + 2)$ (*attention*)

Incrémentation / décrémentation :

$++k$ correspond à $k = k + 1$ ou $k += 1$

$--k$ correspond à $k = k - 1$ ou $k -= 1$

Exemples d'assignations

```
int a = 3;
int b, c;
char d;
b = a;
b = a + 3;
b = b + 1;
c = a * b;
d = 's';
```

a	3						
a	3	b	?	c	?		
a	3	b	?	c	?	d	?
a	3	b	3	c	?	d	?
a	3	b	6	c	?	d	?
a	3	b	7	c	?	d	?
a	3	b	7	c	21	d	?
a	3	b	7	c	21	d	s

Outillage : Commentaires

Il est possible d'écrire des commentaires dans le code pour expliquer ce qu'il fait. Ces commentaires sont ignorés par le compilateur.

Sur une ligne :

```
int tmp = a;
a = b;
b = tmp; // les valeurs de a et b sont échangées
```

Sur plusieurs lignes :

```
/* j'inverse les valeurs de
   a et de b
*/
int tmp = a;
a = b;
b = tmp;
```

Outillage : Sortie (*output*) à l'écran :

```
System.out.print(a);
```

Affiche la valeur de la variable a

```
System.out.println(a);
```

Affiche la valeur de la variable a et passe à la ligne

```
System.out.print("Bienvenue !");
```

Affiche la chaîne de caractères "Bienvenue !"

```
System.out.print(a + 5);
```

Affiche la valeur de l'expression a + 5.

Outils : Entrée (input) au clavier :

Création d'un Scanner (outil de lecture)

```
Scanner s = new Scanner(System.in);
```

Un Scanner est un outil (objet) permettant de lire des valeurs. On le crée à l'aide de "new Scanner(System.in)" pour lire les valeurs sur l'entrée standard (le clavier) et on l'assigne à une variable de type Scanner (ici s).

L'évaluation de l'expression `s.nextInt()`, par exemple, provoque la mise en attente de l'ordinateur d'une valeur entrée par l'utilisateur au clavier. Cette valeur est assignée à la variable `a`.

Lecture d'une valeur entière (int)

```
int a = s.nextInt();
```

Lecture d'une valeur réelle (double)

```
double d = s.nextDouble();
```

Outillage : Structure d'un programme Java

Une instruction simple se termine par un “ ; ”
Les instructions s'exécutent séquentiellement.

Fichier **MonProgramme.java**

```
package intro;

public class MonProgramme {
    public static void main(String[] args) {
        ...instructions...
    }
}
```

Quelques notions que nous détaillerons plus loin :

package = “dossier” dans lequel se trouve le fichier

class = composant d'un programme

public = accessible par tout le monde

public static void main(String[] args) = point d'entrée d'un programme, contient les premières instructions

A priori, la classe porte le même nom que le fichier source (.java)

Exemple complet

Fichier Exemple.java

```
package intro;
```

```
import java.util.Scanner;
```

```
public class Exemple {  
    public static void main(String[] args) {
```

```
        Scanner s = new Scanner(System.in);
```

```
        System.out.print("Entrez deux entiers : ");
```

```
        int a = s.nextInt();
```

```
        int b = s.nextInt();
```

```
        int somme = a + b;
```

```
        System.out.print("La somme vaut ");
```

```
        System.out.println(somme);
```

```
    }
```

```
}
```

importe l'outil Scanner

point d'entrée

création d'un Scanner

lecture de deux entiers et assignations aux variables a et b

calcul de la somme de a et b et assignation à la variable somme

affichage de la somme